

Proyecto 2 - Freesound Audio Tagging

Taller de Aprendizaje Automático

Luciana García

Santiago Márquez

Santiago Rodríguez

5 de julio de 2026

1. Introducción

1.1. Problema y enfoque

El objetivo del proyecto es identificar automáticamente las fuentes sonoras presentes en una grabación. A diferencia de una clasificación tradicional de una sola clase, Freesound Audio Tagging 2019 es un problema *multi-label*: un audio puede contener más de una etiqueta, y la salida esperada por Kaggle es una probabilidad para cada una de las 80 clases. Por este motivo, el modelo se entrena con salidas sigmoides independientes y se evalúa con *label-weighted label-ranking average precision* (lwraps), una métrica de ranking que premia que las clases correctas queden altas en la lista de probabilidades de cada audio.

La solución final se apoya en una hipótesis central: para audio ambiental, la estructura tiempo-frecuencia del espectrograma es más informativa que un resumen tabular global. Por eso el camino de trabajo fue pasar de estadísticas simples de log-mel a redes convolucionales sobre imágenes log-mel, y luego combinar tres ramas que miran el mismo fenómeno con sesgos distintos. La entrega oficial es:

$$0.25 \text{ H100} + 0.375 \text{ G100} + 0.375 \text{ F100}$$

donde H100 es una CNN separable-residual con cabeza densa, G100 agrega normalización global por banda Mel y una cabeza temporal BiGRU, y F100 usa una ventana temporal más larga de 1024 frames. El score privado de Kaggle asociado a este CSV es **0.67126**.

1.2. Metodología general

El trabajo se organizó en cuatro etapas. Primero se analizó el dataset, el desbalance y la métrica. Luego se construyeron caches reproducibles de representaciones log-mel para evitar recalcular audio crudo en cada experimento. En la tercera etapa se compararon familias de modelos y arquitecturas CNN. Finalmente se fijó un sistema de tres ramas a 100 épocas y se evaluaron variantes de ponderación y bagging. La decisión final no tomó el mayor score absoluto, sino el mejor equilibrio entre desempeño, simplicidad y defensa conceptual.

2. Datos, métrica y riesgos

2.1. Estructura del dataset

Se trabajó con tres archivos principales: `train_curated.csv`, `train_noisy.csv` y `sample_submission.csv`. El conjunto curado tiene etiquetas más confiables y fue la base del entrenamiento final; el conjunto ruidoso se analizó como fuente posible, pero no se concatenó directamente porque en experimentos tempranos degradó la validación curada. Además se descartaron seis archivos conocidos como problemáticos antes de entrenar.

Split	Filas	Archivos únicos	Etiquetas promedio/audio
train_curated	4970	4970	1.157
train_noisy	19815	19815	1.211
test	3361	3361	–

Cuadro 1: Tamaño de los conjuntos disponibles. El modelo final entrena sobre el conjunto curado y genera probabilidades para los 3361 audios de test.

Hay 80 clases y 5752 asignaciones de etiquetas en `train_curated`. La mediana de ejemplos por clase es 75, el mínimo observado es 47 y tres clases tienen menos de 50 ejemplos. El desbalance no es extremo por clase individual, pero sí hay una cola de clases raras y una estructura multi-label que hace que la exactitud no sea una métrica adecuada.

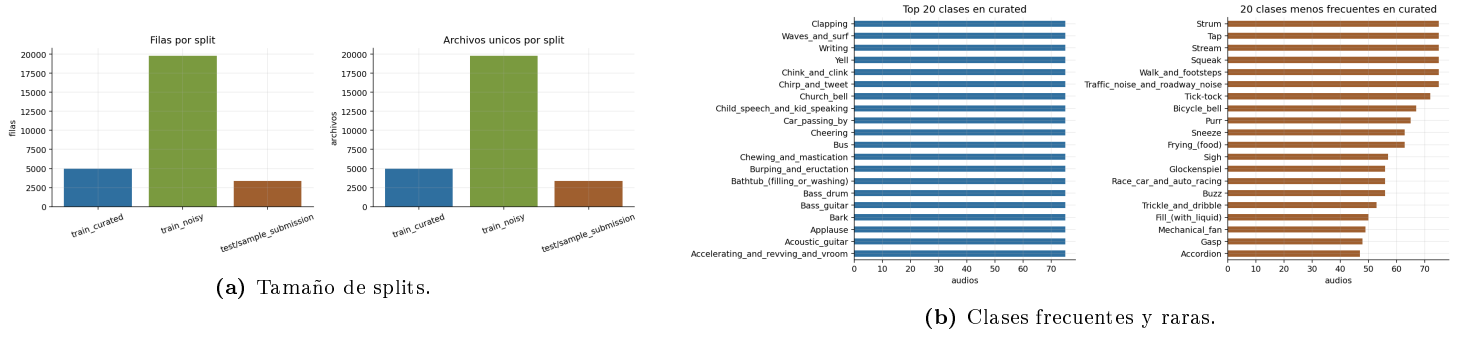


Figura 1: Estructura de datos y desbalance de etiquetas en el conjunto curado.

2.2. Métrica lwrap

Kaggle evalúa con lwrap. Para cada audio se ordenan las clases por score y se mide la precisión acumulada en las posiciones donde aparecen clases verdaderas. Luego se promedia ponderando por la frecuencia de cada clase. Esto favorece modelos que ordenan bien las clases relevantes incluso cuando las probabilidades no están perfectamente calibradas:

$$\text{lwrap} = \sum_{c=1}^C w_c \cdot \frac{1}{N_c} \sum_{i: y_{ic}=1} \text{precision}_{i,c}. \quad (1)$$

Esta métrica justificó trabajar con blends de probabilidades: si dos ramas cometen errores distintos, el promedio puede mejorar el ranking final aunque no cambie la arquitectura interna de cada componente.

3. Representación de audio

3.1. De WAV a log-mel

El preprocesamiento común convierte cada audio en una imagen log-mel:

`.wav ->mono ->44.1 kHz ->MelSpectrogram ->log/dB ->crop/pad ->normalización`

Se usaron 128 bandas Mel. El eje horizontal contiene frames de tiempo; por ejemplo, una entrada 128 x 512 significa 128 bandas de frecuencia perceptual y 512 pasos temporales. Esta representación conserva patrones como ataques, bandas tonales, ruido broadband y repeticiones, que se pierden al colapsar el espectrograma en estadísticas globales.

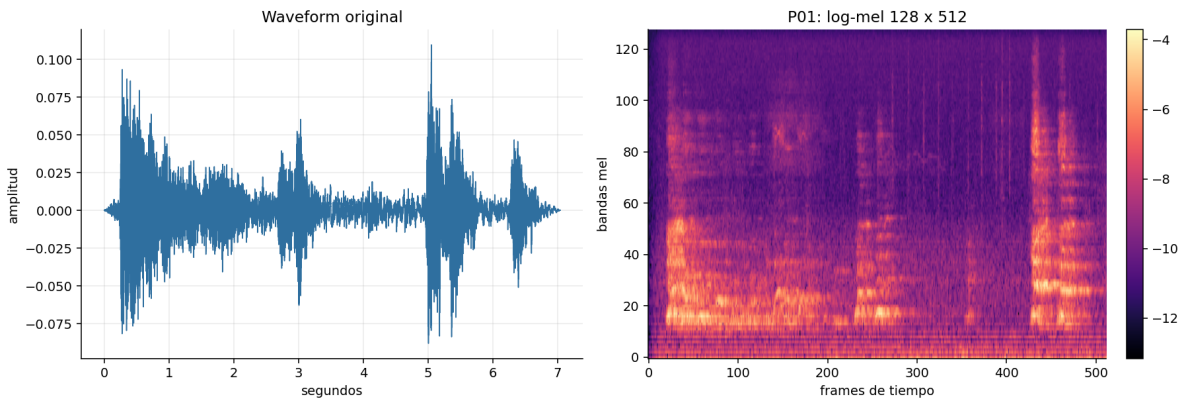


Figura 2: Ejemplo del pasaje desde forma de onda a imagen log-mel. La CNN opera sobre esta estructura tiempo-frecuencia.

3.2. Variantes de preprocesamiento

Como resume la Tabla 2, se probaron tres niveles de representación. P00 resume el log-mel en features tabulares y sirve como baseline barato. P01 conserva la imagen 128 x 512, permitiendo entrenar CNNs. P02 agrega dos ideas que terminan en el final: normalización global por banda Mel y ventana temporal 128 x 1024.

Pipeline	Idea	Private LB	Decisión
P00	Estadísticas tabulares de log-mel; pierde el eje temporal.	0.37607	baseline
P01	Imagen log-mel 128 x 512 para CNN.	0.52257	mantener
P01 + entrenamiento	He, scheduler, BatchNorm, dropout y cabeza densa.	0.64665	mantener
P02 global-mel	Normalización global por banda Mel.	0.66561	mantener
P02 f1024	Contexto temporal de 1024 frames.	0.67025	evidencia pre-final

Cuadro 2: Evolución de las representaciones. El salto grande ocurre al conservar la estructura tiempo-frecuencia para CNNs.

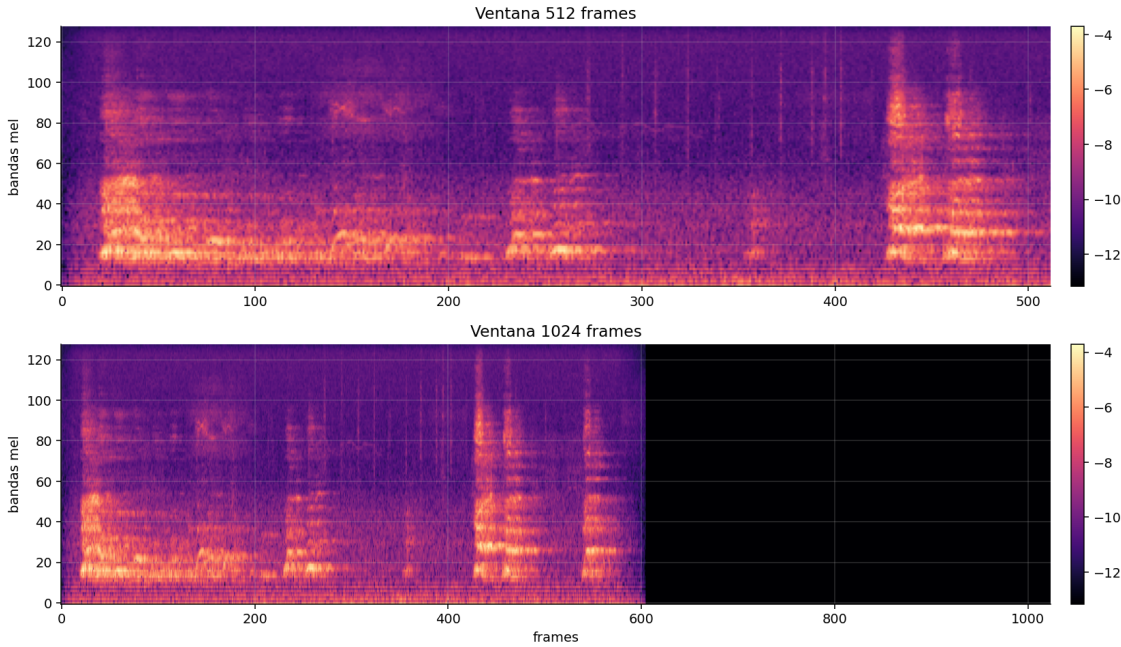


Figura 3: Comparación conceptual entre ventana de 512 frames y ventana de 1024 frames. La segunda conserva más contexto temporal para eventos largos.

4. Modelos y experimentación

4.1. Hipótesis evaluadas

La experimentación se guió por hipótesis concretas, cuyo impacto acumulado se ve en la Figura 4:

- **H1:** resumir log-mel en estadísticas no alcanza, porque colapsa el tiempo. La mejora de 0.37607 a 0.52257 al usar CNN confirma esta hipótesis.
- **H2:** mejorar entrenamiento y regularización de CNNs puede aportar tanto como cambiar arquitectura. La etapa con He, scheduler, BatchNorm, dropout y cabeza densa sube hasta 0.64665.
- **H3:** ramas con sesgos distintos producen errores complementarios. La normalización global Mel y la rama temporal f1024 mejoran los blends.
- **H4:** más complejidad no siempre debe entrar al final. Bagging de ramas temporales alcanza 0.67674, pero agrega modelos internos y reduce claridad.

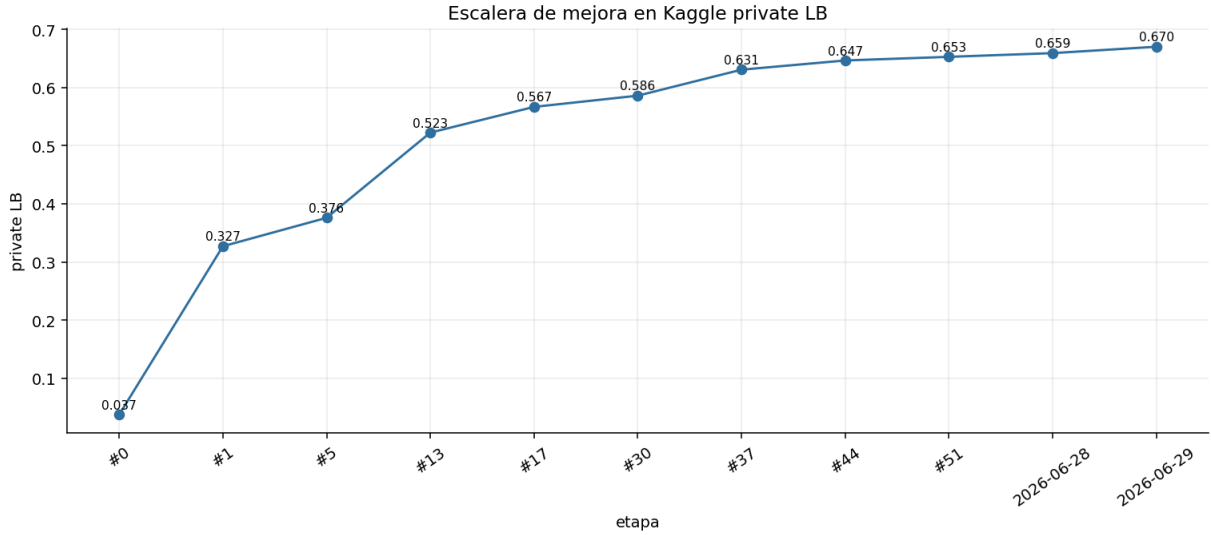


Figura 4: Escalera de mejoras en private leaderboard. Cada salto corresponde a una decisión experimental documentada.

4.2. Arquitecturas comparadas

La comparación incluyó al menos dos familias CNN sobre espectrogramas. La CNN log-mel inicial preserva la imagen 128 x 512 pero usa bloques más simples. La familia final usa bloques separables-residuales: primero una convolución depthwise por canal, luego una convolución 1 x 1 para mezclar canales, BatchNorm y activación. Esta variante es más eficiente y estable, y permitió construir una rama fuerte con cabeza densa.

También se evaluó una variante de transferencia con ResNet50 congelada desde ImageNet. El resultado individual fue débil para usarlo como modelo principal, pero aportó diversidad en blends históricos. Se mantuvo como evidencia de experimentación y no como componente final. Con datos ruidosos ocurrió algo similar: la concatenación directa de `train_noisy` empeoró la validación curada, por lo que se descartó para el final y se dejó como línea futura para preentrenamiento o filtrado.

Prueba	Evidencia	Decisión	Lectura
Transfer ResNet50/ImageNet	débil individual; útil solo como diversidad en blend	no final	la transferencia visual no trasladó suficiente semántica de audio
Concatenar <code>train_noisy</code>	validación curada cae a 0.3567	descartar	el ruido de etiquetas requiere filtrado o preentrenamiento, no mezcla directa
Time reverse / contraste	valid lwlap 0.764–0.771 contra 0.808 de la rama headsep comparable	descartar	algunas augmentations no preservan bien la semántica sonora
CNN separable-residual	valid lwlap 0.8078 y Private LB 0.65289	mantener	mejora capacidad sin perder una explicación simple

Cuadro 3: Pruebas asociadas a requisitos de la consigna que no entraron directamente al modelo final.

5. Modelo final

5.1. Tres ramas complementarias

La entrega oficial combina tres componentes entrenados a 100 épocas con semilla 42. Las tres usan log-mel, pero no son redundantes.

Rama	Peso	Entrada	Normalización	Rol
<code>separable_headsep</code>	0.25	128 x 512	por clip	patrones locales tiempo-frecuencia con CNN separable-residual y cabeza densa
<code>globalmel_sep_temporal</code>	0.375	128 x 512	global Mel	escala estable por banda y evolución temporal con BiGRU
<code>sep_temporal_f1024</code>	0.375	128 x 1024	por clip	contexto temporal largo con CNN separable-residual y BiGRU

Cuadro 4: Componentes del blend final. Los pesos suman 1 y dan mayor peso conjunto a las ramas temporales.

La rama `separable_headsep` termina en pooling global y una cabeza densa, por lo que resume la activación espacial final. Las ramas temporales no colapsan tan pronto el eje de tiempo: la CNN extrae mapas tiempo-frecuencia y una BiGRU bidireccional modela la secuencia resultante antes de clasificar.

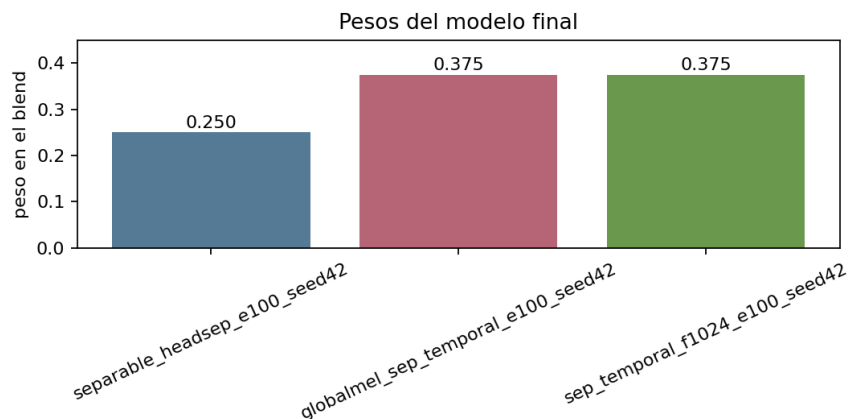


Figura 5: Pesos del blend final. La reponderación baja la rama puramente convolucional y sube las dos ramas temporales.

5.2. Regularización e hiperparámetros

La regularización no depende de una sola técnica. Las tres ramas usan `head_dropout=0.3` y `BatchNorm` en los bloques convolucionales. Durante entrenamiento se aplica `SpecAugment` sobre el log-mel: corrimiento temporal, máscara temporal y máscara de frecuencia. Las ramas temporales usan `AdamW` con `weight_decay=1e-4`; la rama `headsep` usa `Adam`, por lo que en el trainer actual no recibe L2 efectiva. Además, la pérdida es `BCEWithLogitsLoss` con `pos_weight` calculado desde el conjunto curado para compensar desbalance por clase.

Se dejaron apagadas opciones que agregaban complejidad sin evidencia suficiente: `block_dropout=0`, `early_stopping=0`, inversión temporal y cambio aleatorio de contraste. La configuración final es por lo tanto simple de explicar: tres ramas, 100 épocas, augmentation controlada y blend lineal.

Para controlar subajuste y sobreajuste se observó simultáneamente pérdida de entrenamiento, `lwlrap` de validación y `leaderboard`. En la rama separable-residual con cabeza densa, por ejemplo, la pérdida bajó de 0.601 a 0.023 y la validación subió hasta 0.8078 alrededor de la época 56; luego osciló levemente, señal de que seguir entrenando esa corrida ya aportaba poco al holdout. En las corridas finales se priorizó *full-train* a 100 épocas porque la evidencia de Kaggle de la Tabla 5 mostró mejora al usar toda la data curada y no únicamente el split local.

6. Resultados y selección final

La ronda a 100 épocas mostró que la arquitectura conceptual seguía mejorando al entrenar más. El sistema de tres ramas con pesos iguales alcanzó 0.67055, y la reponderación 0.25/0.375/0.375 subió a 0.67126. Luego se probaron baggings internos sobre ramas temporales: el mejor llegó a 0.67674, pero se descartó como entrega principal por depender de más modelos y ser menos directo para defender en el tiempo disponible. La diferencia a favor del mejor experimento es 0.00548, pero la entrega oficial mantiene tres componentes físicos no baggeados y una fórmula de pesos simple.

Candidato	Private LB	Estado	Razón
3-way original	0.66649	referencia	tres ramas previas con pesos iguales
3-way e100 equal	0.67055	mantiene	misma idea entrenada a 100 épocas
3-way e100 weighted	0.67126	final oficial	tres componentes físicos y pesos simples
globalmel bagged	0.67528	experimento	mejora parcial con bagging interno
globalmel + f1024 bagged	0.67674	mejor score	más complejo; no se elige como entrega

Cuadro 5: Comparación de variantes alrededor del modelo final.

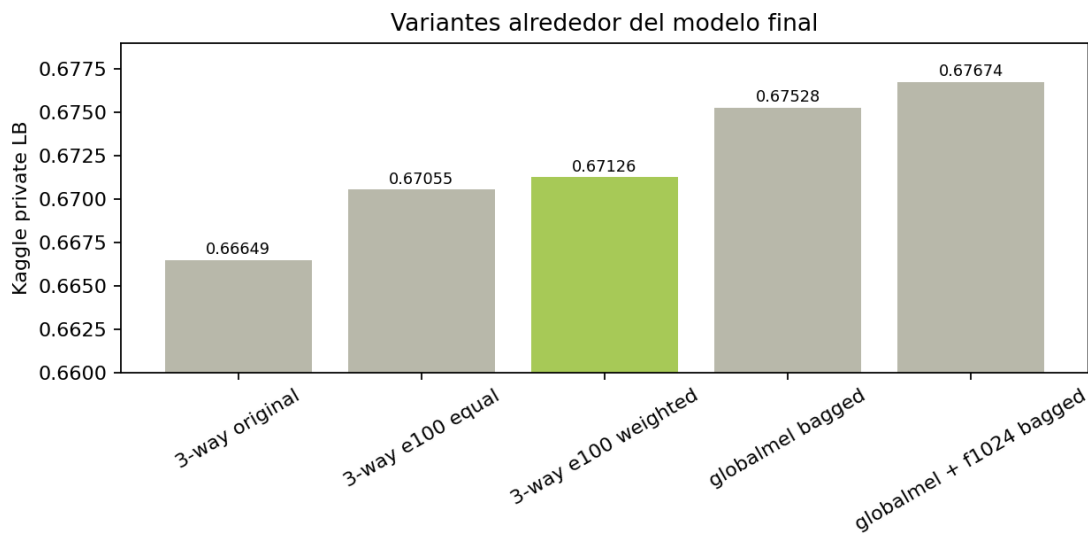


Figura 6: Experimentos finales. El mejor score absoluto queda documentado, pero la entrega oficial prioriza trazabilidad y simplicidad.

7. Entrega reproducible

El código de entrega queda en `proyecto_actual/codigo/`. El notebook `pipeline_final_taller_2.ipynb` recompone el blend, valida columnas, orden de archivos, rango de probabilidades y hash SHA-256 del CSV oficial:

4247ab9ff6398fbb1b6af223629d004265e27bb6cbccabf53ec4969a96c61cab

El mismo notebook contiene los comandos pesados para reconstruir caches log-mel y entrenar las tres ramas si se activa `RUN_HEAVY_STEPS=True`. En modo entrega se ejecuta rápido usando los artefactos oficiales ya generados, lo que facilita verificar la submission sin exigir una GPU.

7.1. Lectura de predicciones de validación

Para interpretar las salidas se revisaron ejemplos del ensamble final con fine-tuning sobre `train_curated` + `train_noisy`. Estos casos provienen del split local usado como control de degradación: no son una validación independiente, porque los checkpoints base fueron entrenados con todo `train_curated`, pero sirven para inspeccionar si el ranking de clases del modelo final es coherente.

Audio	Etiquetas reales	Top-5 predicho	Lectura
cce07823.wav	Fill (with liquid), Gurgling, Sink, Water tap	Fill 0.993; Water tap 0.980; Sink 0.977; Gurgling 0.976; Trickle 0.968	las cuatro etiquetas reales quedan arriba; la clase extra también es de agua
38b700ca.wav	Applause, Cheering, Crowd	Applause 1.000; Crowd 0.999; Cheering 0.999; Harmonica 0.073; Run 0.036	ranking correcto y muy concentrado en el evento de público
a61153d2.wav	Acoustic guitar, Strum	Acoustic guitar 1.000; Strum 1.000; Bass guitar 0.020; Electric guitar 0.005; Gong 0.003	identifica instrumento y acción; los falsos positivos son instrumentos cercanos
c197ef4f.wav	Bicycle bell	Bicycle bell 1.000; Glockenspiel 0.180; Chink/clink 0.067; Microwave 0.029; Shatter 0.023	top-1 correcto; las alternativas son sonidos metálicos breves, acústicamente plausibles

Cuadro 6: Ejemplos de interpretación del ensamble final con noisy fine-tune.

8. Conclusiones

El resultado principal del proyecto es que la representación importa tanto como la arquitectura. El salto más claro aparece al pasar de estadísticas globales de log-mel a imágenes log-mel para CNNs. Luego, la mejora sostenida viene de agregar diversidad controlada: normalización global por banda Mel, lectura temporal con BiGRU y mayor contexto temporal. La submission oficial no es el máximo score exploratorio, sino la variante más defendible: conserva tres componentes conceptuales, evita bagging interno adicional y alcanza un private LB competitivo de 0.67126.

Como trabajo futuro, el uso de `train_noisy` podría retomarse con una estrategia más cuidadosa, por ejemplo preentrenamiento con etiquetas débiles, filtrado por confianza o fine-tuning progresivo. También sería natural mejorar interpretabilidad visual revisando ejemplos de validación donde cada rama asigna alta probabilidad a clases distintas.

Referencias

- [1] Instituto de Ingeniería Eléctrica, FING, Udelar. *Proyecto 2 - Freesound Audio Tagging*, Taller de Aprendizaje Automático, 2026.
- [2] Kaggle. *Freesound Audio Tagging 2019*. <https://www.kaggle.com/c/freesound-audio-tagging-2019>
- [3] Material del curso Taller de Aprendizaje Automático: clases de redes convolucionales, regularización, transferencia y audio.
- [4] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly.